Πανεπιστήμιο Κύπρου
Τμήμα Πληροφορικής

ΕΠΛ445: Ψηφιακή Επεξεργασία Εικόνας

Εργαστήριο

Εργαστηριακή Άσκηση 4

Ομάδα:

Stefanos Panteli UC1065916
Evangelia Krigkou UC1067400
Rafael Chrysanthou UC1064747

Ημερομηνία Παράδοσης
13 Μαρτίου 2026

1. Εισαγωγή - Στόχος

Να κατανοήσουμε πως δουλεύει ο αλγόριθμος συμπίεσης JEPG, ο οποίος έχει απώλεια πληροφορίας. Στα πλαίσια της άσκησης έπρεπε να ελέγξουμε διάφορα factors συμπίεσης, το οποίο είναι ανάλογο με την απώλεια. Για να εξετάσουμε την απώλεια, χρησιμοποιήσαμε διάφορες μετρικές, όπως, MSE, PSNR, SSIM, αλλά και το Compression Ratio που ελέγχει πόσο πιο μικρή είναι η νέα εικόνα σε σύγκριση με την αρχική.

2. Μεθοδολογία

1. Διάβασμα εικόνας σε κλίμακα γκρι

```
image = cv2.imread('input/dog.png', 0)
print('Original Shape:', image.shape)
# Resize so it is a multiple of 8
image = cv2.resize(image, (image.shape[1] // 8 * 8, image.shape[0] // 8 * 8))
print('Cropped Shape:', image.shape)
```

2α. Υλοποίηση αλγόριθμου JPEG για κωδικοποίηση

A. Σπάσιμο πίνακα σε blocks 8x8

B.

```
def block_partition(self, I, block_size = 8):
    vector_blocks = []
    iHeight, iWidth = I.shape

    # Image partitioned into block_size x block_size blocks
    for startY in range(0, iHeight, block_size):
      for startX in range(0, iWidth, block_size):
        block = I[
          # Take the block_size sized block
          startY : startY + block_size,
          startX : startX + block_size
        ]
        vector_blocks.append(block)

    return vector_blocks
```

C. Εκτέλεση DCT στο κάθε block

```
def dct(self, block):
    # Apply DCT to the block casted to a float
    return cv2.dct(np.float32(block))
```

D. Εκτέλεση κβαντοποίησης

```
def quantization(self, dct_block, factor):
    # Multiply the q_table by the factor and divide each pixel by q value
    q_table = np.multiply(self.q_table, factor)
    return np.floor(np.divide(dct_block, q_table) + 0.5).astype(np.int32)
```

E. Αναδιάταξη Zig-Zag

```
def rearrange_zig_zag(self, q_block, block_size= 8):
    zig_zag_sequence = np.zeros(block_size**2, dtype= q_block.dtype)

    # Populate the 1D array based on the zig_zag_table mapping
    for i in range(block_size):
        for j in range(block_size):
            zig_zag_sequence[self.zig_zag_table[i, j]] = q_block[i, j]

    return zig_zag_sequence
```

F. DPCM για τις συνιστώσες DC

```
def dpcm(self, vectors_zigzag):
    e = []
    # leave first value of first vector as it is
    e.append(vectors_zigzag[0][0])
    for k in range(1, len(vectors_zigzag)):
        # Apply DPCM to the rest of the DC compontents
        e.append(vectors_zigzag[k][0] - vectors_zigzag[k - 1][0])

    return e
```

G. RLC για τις συνιστώσες AC

```
def rlc(self, vectors_zigzag):
    def _convert(block_ac_values):
        to_return = []
        zeroes = 0
        for i in range(len(block_ac_values)):
            # Count consecutive zeroes
            if block_ac_values[i] == 0:
                zeroes += 1
            else:
                # And append a (zeroes, next non-zero value)
                to_return.append((zeroes, block_ac_values[i]))
                # Reset the zero counter
                zeroes = 0
        to_return.append((0, 0))
        return to_return

    return [
        # Conver all 1D blocks into RLC lists
        _convert(ac[1:])
        for ac in vectors_zigzag
    ]
```

H. Huffman Encoding

```
def huffman_encode(self, values):
    symbols_to_process = values
    # For the AC
    if values and isinstance(values[0], list) and values[0] and
                                    isinstance(values[0][0], tuple):
        processed_ac_symbols = []
        for sublist in values:
            for skip, val in sublist:
                # Parse any possible (skip, value) tuple
                if isinstance(val, (np.float32, np.float64, np.floating)):
                    processed_ac_symbols.append((skip, float(val)))
                elif isinstance(val, (np.int32, np.int64, np.integer)):
                    processed_ac_symbols.append((skip, int(val)))
                else:
                    processed_ac_symbols.append((skip, val))
        symbols_to_process = processed_ac_symbols
    # For the DC, just making sure they are all floats
    elif all(isinstance(v, (int, float, np.integer, np.floating)) for v in values):
        symbols_to_process = [float(val) for val in values]

    # Compute the percentages
    counter = collections.Counter(symbols_to_process)
    probs = [(key, np.float32(value)) for key, value in counter.items()]
    # Run the Huffman encoding
    symbols = Huffman3.makenodes(probs)
    root = Huffman3.iterate(symbols)
    return Huffman3.encode(symbols_to_process, symbols), root
```

I. Όλα μαζί για κωδικοποίηση

```
def encode(self, I, factor, block_size = 8):
    blocks = self.block_partition(I, block_size)
    dct_blocks = [self.dct(block) for block in blocks]
    q_blocks = [self.quantization(block, factor) for block in dct_blocks]
    zig_blocks = [
        self.rearrange_zig_zag(block, block_size)
        for block in q_blocks
    ]

    dc_components = self.dpcm(zig_blocks)
    ac_components = self.rlc(zig_blocks)

    dc_symbols, dc_root = self.huffman_encode(dc_components)
    ac_symbols, ac_root = self.huffman_encode(ac_components)

    return (dc_symbols, dc_root, ac_symbols, ac_root)
```

2β. Υλοποίηση αλγόριθμου JPEG για αποκωδικοποίηση

Για εξοικονόμηση χώρου, μια και είναι πολύ όμοιο με την κωδικοποίηση, ο κώδικας βρίσκεται στο αρχείο *hw5.ipynb* και εδώ θα βάλουμε τον κώδικα για το decode που χρησιμοποιεί όλες τις συναρτήσεις.

```
def decode(self, dc_encoded, dc_root, ac_encoded, ac_root,
            factor, original_shape, block_size = 8):

    decoded_dc_values = self.huffman_decode(dc_encoded, dc_root)
    decoded_ac_tuples = self.huffman_decode(ac_encoded, ac_root)

    decoded_dc_values = [float(x) for x in decoded_dc_values]

    dc_components = self.inv_dpcm(decoded_dc_values)
    ac_components = self.inv_rlc(decoded_ac_tuples, block_size)

     combined_blocks_1d = self.combine(
            dc_components, ac_components, block_size
     )
    inv_zig_zag_blocks = [
            self.inv_zig_zag(block, block_size)
            for block in combined_blocks_1d
   ]
    inv_quantized_blocks = [
            self.inv_quantization(block, factor)
            for block in inv_zig_zag_blocks
   ]
    inv_dct_blocks = [self.inv_dct(block) for block in inv_quantized_blocks]

    reconstructed = self.merge_blocks(inv_dct_blocks, original_shape, block_size)
    return self.bound(reconstructed)
```

4. Υπολογισμός μετρικών

```
def compute_metrics(I, comp_I, in_path, out_path):
  MSE = skimage.metrics.mean_squared_error(I, comp_I)
  PSNR = skimage.metrics.peak_signal_noise_ratio(I, comp_I)
  SSIM = skimage.metrics.structural_similarity(
      I, comp_I, data_range= comp_I.max() - comp_I.min()
  )
  CR = os.path.getsize(in_path) / os.path.getsize(out_path)
  return {
    'MSE': round(float(MSE), 2),
    'PSNR': round(float(PSNR), 2),
    'SSIM': round(float(SSIM), 2),
    'CR': round(float(CR), 2)
  }
```

5. Χρήση του αλγορίθμου σε εικόνα

```
metrics = {}
for factor in q_factors:
  components = EncodeJPEG().encode(image, factor)
  comp_I = DecodeJPEG().decode(*components, factor, image.shape)

  cv2.imwrite(f'output/compressed_{factor}.jpg', comp_I)
  metrics[factor] = compute_metrics(
        image, comp_I, 'input/dog.png', f'output/compressed_{factor}.jpg'
  )
  for metric, value in metrics[factor].items():
    print(f'{metric}: {value}')
  _plot(image, comp_I, factor)

  print()
```

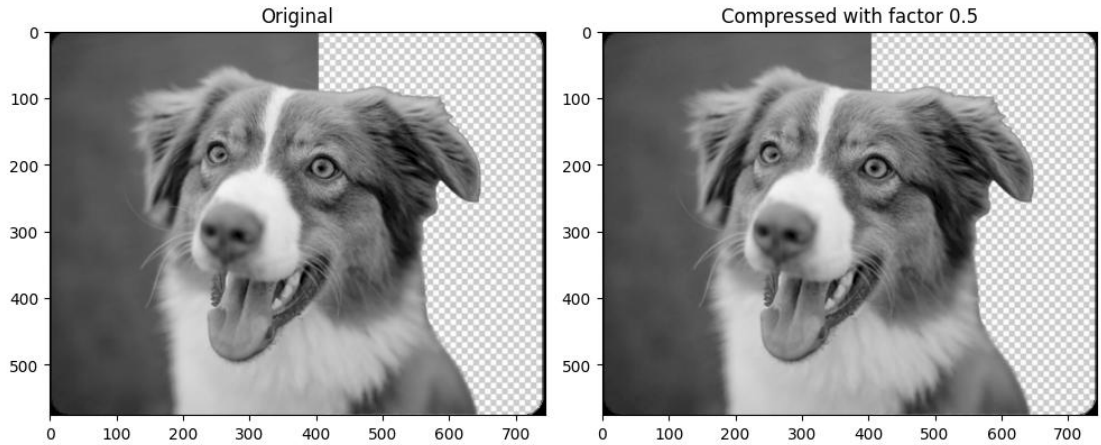
6. Γράφημα

```
def _plot(I, comp_I, factor):
    plt.figure(figsize=(10, 10))
    plt.subplot(121)
    plt.title(f'Original')
    plt.imshow(I, cmap= 'gray')
    plt.subplot(122)
    plt.title(f'Compressed with factor {factor}')
    plt.imshow(comp_I, cmap= 'gray')
    plt.tight_layout()
    plt.show()
```

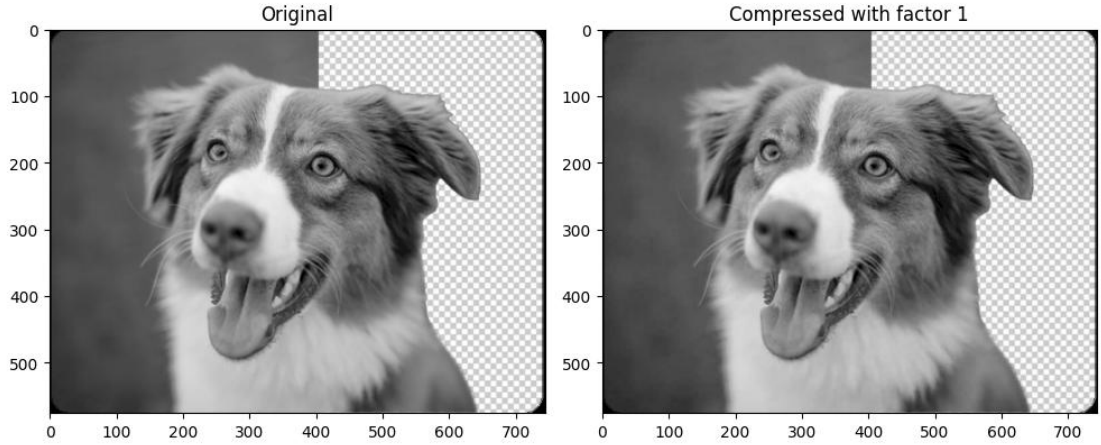
3. Αποτελέσματα

Αρχική εικόνα ελάχιστα cropped ώστε οι διαστάσεις να είναι πολλαπλάσια του 8

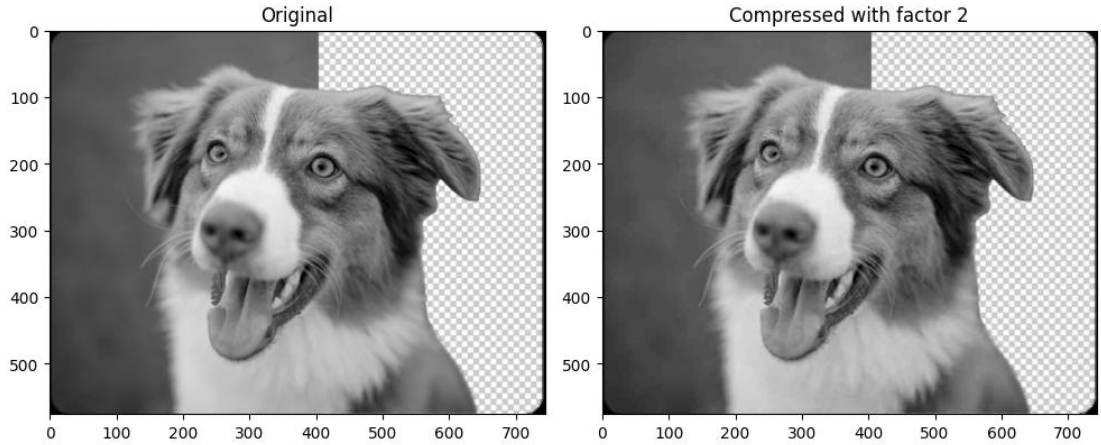
Οι επόμενες εικόνες δείχνουν την original (αυτή πάνω) στα αριστερά και την compressed στα δεξιά. Κάναμε χρήση πιο μεγάλων factor για να φανεί πολύ ξεκάθαρα η διαφορά της τιμής του factor στην συμπιεσμένη εικόνα. Από κάτω από τα γραφήματα, φαίνονται οι μετρικές.



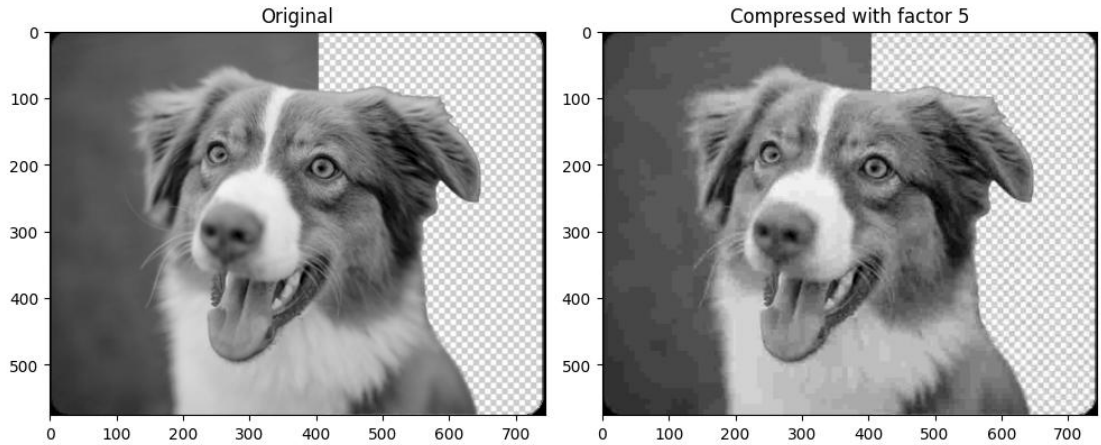
MSE: 5.64 PSNR: 40.62 SSIM: 0.98 CR: 6.89



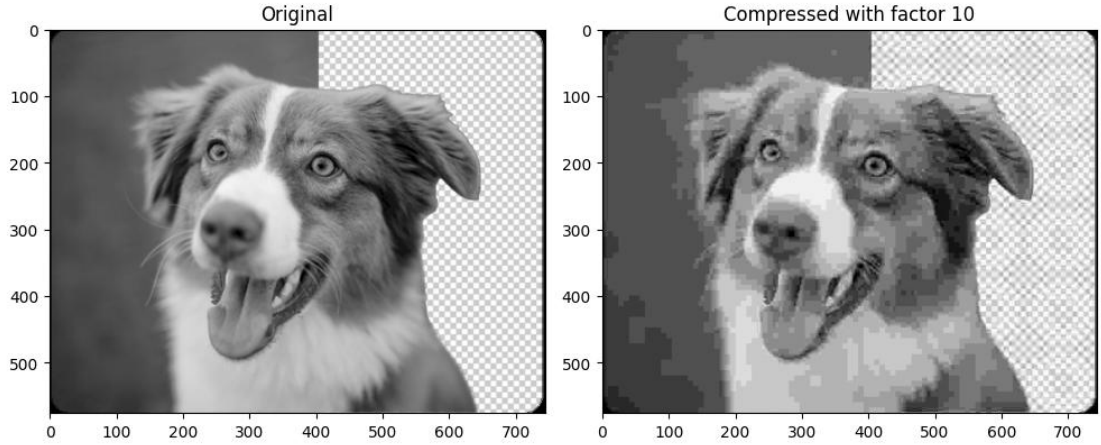
MSE: 10.25 PSNR: 38.02 SSIM: 0.97 CR: 8.06



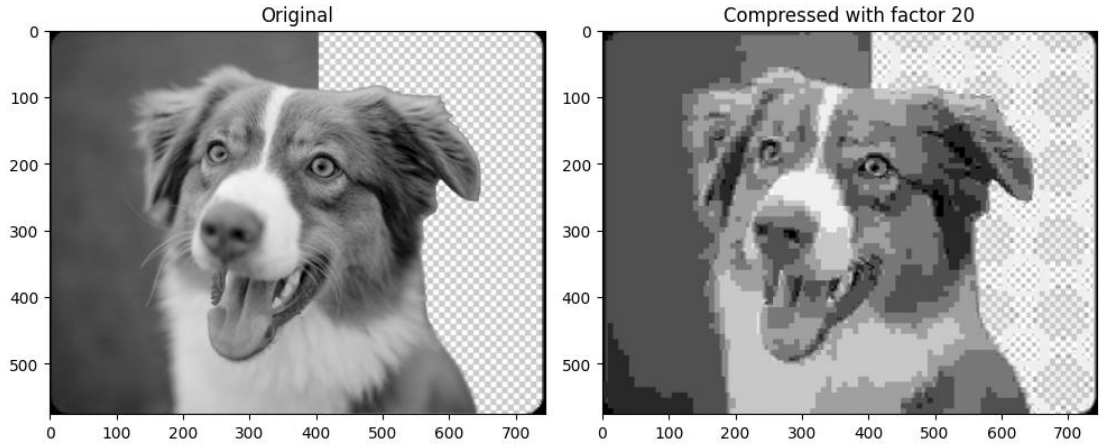
MSE: 19.82 PSNR: 35.16 SSIM: 0.94 CR: 9.77



MSE: 56.2 PSNR: 30.63 SSIM: 0.87 CR: 13.45

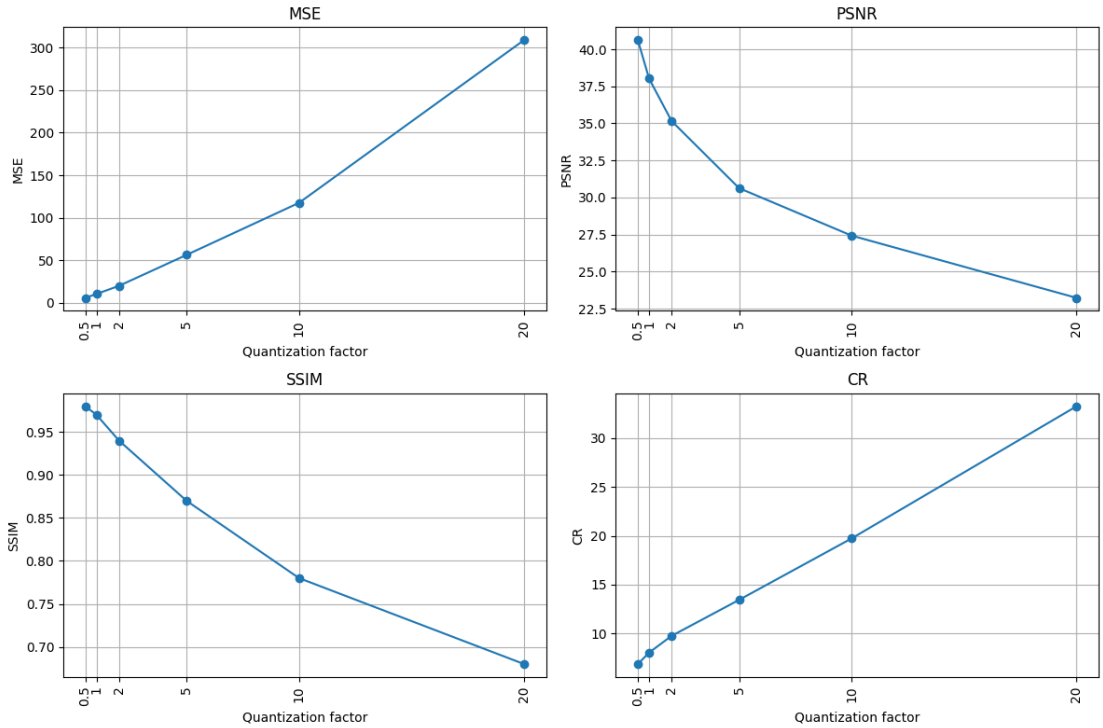


MSE: 117.49 PSNR: 27.43 SSIM: 0.78 CR: 19.73



MSE: 308.98 PSNR: 23.23 SSIM: 0.68 CR: 33.24

Φαίνεται ξεκάθαρα ότι όσο πιο μεγάλη η τιμή του factor τόσο πιο blocky και η συμπιερμένη εικόνα. Ακολουθέι ανάλυση των μετρικών.



Φαίνεται καθαρά ότι καθός αυξάνεται το factor:

1. Το MSE αυξάνεται σχεδόν ανάλογα (αυξάνεται λίγο πιο γρήγορα).
2. Το PSNR μειώνεται, αλλά η μείωση μεταξύ διαδοχικά factor values μειώνεται.
3. Το SSIM μειώνεται.
4. Το CR αυξάνεται ανάλογα.

4. Συμπεράσματα - Σχολιασμός Αποτελεσμάτων

Ο αλγόριθμος lossy JPEG Compression είναι ένας αλγόριθμος που μειώνει κατά πολύ το μέγεθος μιας εικόνας, και για χαμηλές τιμές του factor η εικόνα στο μάτι δεν έχει πολλές διαφορές, όπως γίνεται αντιληπτό και από το SSIM. Για *factor = 2* το SSIM είναι μεγαλύτερο του 0.94, άρα είναι αποδεκτό loss, ενώ ταυτόχρονα επιτυγχάνει Compression Ratio κοντά στο 10 (!). Όσο αυξάνεται το factor, τα blocks γίνονται αντιληπτά αφού χάνεται πάρα πολύ πληροφορία. Για *factor = 20* Ο σκύλος είναι ακόμα αντιληπτός, παρά που χάθηκαν όλες οι λεπτομέρειες λόγω του 33 Compression Ratio.